

SINGLE CYCLE RISC-V IMPLEMENTATION USING VERILOG

PREPARED BY:

Dharmi Patel
Nensi Thummar

UNDER:

Keval Jadav
Manav Jani



MAY 2026
(SUMMER INTERNSHIP)

Contents

1. ABSTRACT	4
2. INTRODUCTION	4
2.1 Introduction to RISC-V	4
2.2 Single-Cycle Processor Architecture	5
2.3 Need for Single-Cycle Design	5
3. OBJECTIVES	5
4. SYSTEM ARCHITECTURE	6
4.1 Overall Block Diagram	6
4.2 Single-Cycle Instruction Execution	7
5. DESIGN MODULE	8
5.1 Program Counter (PC)	8
5.2 Instruction Memory	8
5.3 Instruction Decode Unit	8
5.4 Register File	8
5.5 Sign Extension Unit	8
5.6 Arithmetic Logic Unit (ALU)	9
5.7 Data Memory	9
5.8 Control Unit	9
5.9 ALUSrc Multiplexer	10
5.10 Result Multiplexer	10
5.11 WD3 Multiplexer	10
5.12 PC Multiplexer	10
5.13 PC+4 Adder	10
5.14 PC Target Adder	10
6. CONTROL SIGNALS	11
6.1 Control Signals	12
7. MUX ENCODING	13
7.1 ResultSrc Encoding	13
7.2 PCSrc Encoding	13
7.3 ImmSrc Encoding	13
8. INSTRUCTION FORMATS OF RISC-V	14
8.1 R-Type Instructions	14
8.2 I-Type Instructions	14
8.3 S-Type Instructions	15
8.4 B-Type Instructions	15

8.5	U-Type Instructions.....	15
8.6	J-Type Instructions.....	15
9.	DATAPATH OPERATION FOR INSTRUCTIONS	16
9.1	R-Type Instruction Execution	16
9.2	I-Type Instruction Execution	16
9.3	Load and Store Instruction Execution.....	16
9.4	Branch Instruction Execution.....	16
9.5	Jump Instruction Execution	17
10.	TESTBENCH DESIGN	17
11.	SIMULATION RESULTS	18
12.	COMPARISION WITH MULTICYCLE AND PIPELINE	19
13.	ADVANTAGES	20
14.	LIMITATIONS	20
15.	APPLICATIONS.....	20
16.	CHALLENGE FACED	20
17.	LEARNING OUTCOMES	20
18.	FUTURE ENHANCEMENTS	20
19.	CONCLUSION	21
20.	REFERENCES.....	21

1. ABSTRACT

This project presents the design and implementation of a 32-bit Single-Cycle RISC-V Processor using Verilog Hardware Description Language (HDL). RISC-V is a modern open-source Instruction Set Architecture (ISA) that provides a simple, modular, and flexible platform for processor development. The single-cycle architecture executes every instruction in a single clock cycle, making the processor easier to design, understand, and verify while demonstrating the fundamental principles of computer architecture.

The implemented processor is based on the RV32I instruction set and consists of essential modules such as the Program Counter (PC), Instruction Memory, Register File, Control Unit, Sign Extension Unit, Arithmetic Logic Unit (ALU), Data Memory, multiplexers, and address generation units. These modules work together to perform instruction fetch, decode, execute, memory access, and write-back operations within one clock cycle. The processor supports arithmetic, logical, load, store, branch, and jump instructions through appropriate control signal generation and datapath control.

To demonstrate the operation of the processor, datapath execution for different instruction formats and instruction types was analyzed. The design was modelled in Verilog HDL and verified using a dedicated testbench. Various test cases were performed to validate arithmetic computations, memory operations, branching functionality, and register updates. Simulation results confirmed the correct execution of supported RV32I instructions and proper interaction among all processor modules.

The project provides a clear understanding of processor datapath design, control signal generation, instruction execution, and RISC-V architecture fundamentals. It also serves as a foundation for advanced processor implementations such as multi-cycle and pipelined RISC-V architectures.

2. INTRODUCTION

A processor executes instructions stored in memory to perform arithmetic, logical, control, and data transfer operations. The efficiency of a processor depends on its architecture and the method used to execute instructions. RISC-V is a modern open-source Instruction Set Architecture (ISA) that provides a simple and flexible platform for processor design. One of the fundamental implementations of a processor is the single-cycle architecture, where every instruction is executed completely within a single clock cycle. This project implements a 32-bit Single-Cycle RISC-V processor using Verilog HDL and demonstrates the execution of arithmetic, logical, memory, branch, and jump instructions through a unified datapath and control structure.

2.1 Introduction to RISC-V

RISC-V is an open-source Reduced Instruction Set Computer (RISC) Instruction Set Architecture developed at the University of California, Berkeley. It provides a simple, modular, and extensible instruction set that can be freely used, modified, and implemented without licensing restrictions.

RISC-V follows the RISC design philosophy, which emphasizes simple instructions that can be executed efficiently by hardware. The architecture supports various instruction formats including R-Type, I-Type, S-Type, B-Type, U-Type, and J-Type instructions. Due to its openness, flexibility, and scalability, RISC-V has become widely adopted in education, research, embedded systems, and commercial processor development.

2.2 Single-Cycle Processor Architecture

A Single-Cycle Processor is a processor architecture in which every instruction is executed within one clock cycle. All stages of instruction execution, including Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB), are completed during the same clock period.

The processor consists of several functional units such as the Program Counter (PC), Instruction Memory, Register File, Control Unit, Sign Extension Unit, Arithmetic Logic Unit (ALU), Data Memory, multiplexers, and adders. These modules work together through a common datapath to execute instructions and generate the required results. Since each instruction completes in a single cycle, the control logic becomes relatively simple and straightforward to implement.

2.3 Need for Single-Cycle Design

In processor design, understanding the complete flow of instruction execution is essential before moving to more advanced architectures. The Single-Cycle Processor provides a simple and effective platform for studying processor operation and datapath design.

The need for a Single-Cycle design includes:

- Simplicity of hardware implementation
- Easy understanding of instruction execution flow
- Simple control unit design
- Useful for educational and research purposes
- Serves as a foundation for multi-cycle and pipelined processors
- Facilitates verification and debugging of processor functionality

Although single-cycle processors have lower performance compared to pipelined architectures, they provide a clear understanding of how different processor components interact to execute instructions.

3. OBJECTIVES

The objectives of this project are:

- To understand the working of the RISC-V Instruction Set Architecture (ISA).
- To design and implement a 32-bit Single-Cycle RISC-V processor using Verilog HDL.
- To develop essential modules such as the ALU, Register File, Control Unit, Instruction Memory, and Data Memory.
- To execute arithmetic, logical, load, store, branch, and jump instructions.
- To verify processor functionality using simulation and testbench analysis.
- To study datapath design and instruction execution in a Single-Cycle processor.

4. SYSTEM ARCHITECTURE

The proposed processor is a 32-bit Single-Cycle RISC-V processor based on the RV32I instruction set architecture. In this design, every instruction is fetched, decoded, executed, and completed within a single clock cycle. The processor consists of two major sections: the Datapath and the Control Unit. The Datapath performs data processing and instruction execution, while the Control Unit generates the required control signals for different instruction types.

The processor supports arithmetic, logical, immediate, load, store, branch, jump, and upper-immediate instructions. The hardware modules used in the design are:

- Program Counter (PC)
- Instruction Memory
- Instruction Decode Unit
- Register File
- Sign Extension Unit
- Arithmetic Logic Unit (ALU)
- Data Memory
- Control Unit
- ALUSrc Multiplexer
- Result Multiplexer
- WD3 Multiplexer
- PC Multiplexer
- PC+4 Adder
- PC Target Adder

4.1 Overall Block Diagram

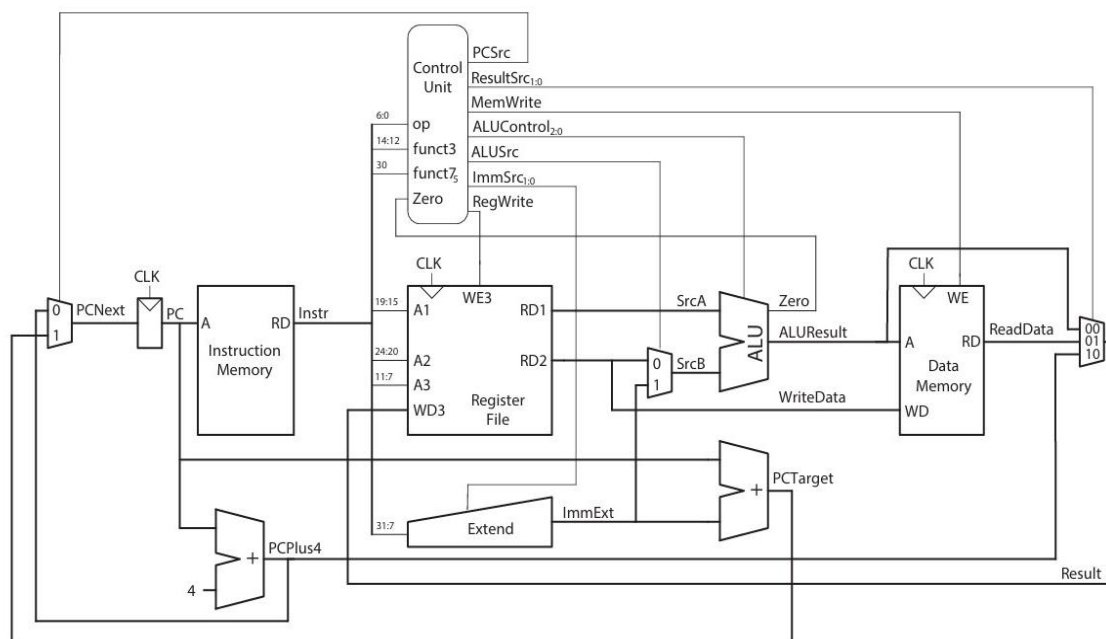


Fig.1: Single Cycle RISC-V Block Diagram

The overall architecture consists of a Control Unit and a Datapath connected through various control and data signals. The Program Counter (PC) supplies the address of the current instruction to the Instruction Memory. The fetched instruction is decoded to obtain the opcode, source registers, destination register, function fields, and immediate values.

The Register File provides operand data to the ALU. Depending on the instruction type, the ALU performs arithmetic, logical, comparison, or address calculation operations. For load and store instructions, the Data Memory is accessed using the address generated by the ALU. The Result Multiplexer selects the appropriate result and writes it back to the destination register. For branch and jump instructions, the PC is updated using either the PC+4 Adder, PC Target Adder, or ALU-generated address.

4.2 Single-Cycle Instruction Execution

In a single-cycle architecture, all stages of instruction execution are completed during one clock cycle.

1. **Instruction Fetch (IF)**
The Program Counter provides the address of the instruction, which is fetched from the Instruction Memory.
2. **Instruction Decode (ID)**
The instruction fields are decoded and source operands are read from the Register File. The Control Unit generates the required control signals, and immediate values are generated by the Sign Extension Unit.
3. **Execute (EX)**
The ALU performs arithmetic, logical, comparison, branch, or address calculation operations based on the instruction type.
4. **Memory Access (MEM)**
For load instructions, data is read from Data Memory. For store instructions, data is written into Data Memory.
5. **Write Back (WB)**
The final result, memory data, immediate value, or PC+4 value is selected and written back into the destination register.

Since all these operations occur within a single clock period, each instruction completes execution in one clock cycle before the next instruction begins.

5. DESIGN MODULE

5.1 Program Counter (PC)

The Program Counter (PC) stores the address of the current instruction being executed. During each clock cycle, the PC is updated with the address of the next instruction. Under normal execution, the PC increments by 4 to fetch the next sequential instruction. For branch and jump instructions, the PC is updated with the target address generated by the processor.

5.2 Instruction Memory

The Instruction Memory stores the program instructions executed by the processor. During the Instruction Fetch stage, the PC provides the address of the instruction to be fetched. The corresponding 32-bit instruction is then supplied to the datapath for decoding and execution.

Functions:

- Store program instructions
- Provide instructions based on the PC value
- Support sequential instruction fetching

5.3 Instruction Decode Unit

The Instruction Decode Unit extracts different fields from the fetched instruction, including opcode, source registers, destination register, function fields, and immediate values. These fields are used by the Control Unit and Datapath to determine the required operation.

Functions:

- Decode instruction fields
- Extract register addresses
- Identify instruction type
- Supply control and datapath information

5.4 Register File

The Register File stores temporary data required during instruction execution. It consists of 32 general-purpose registers, each having a width of 32 bits. Register x0 is permanently fixed to zero .

The Register File provides:

- Two read ports and one write port
- Simultaneous read and write operations

5.5 Sign Extension Unit

Different RISC-V instructions contain immediate values of varying sizes. The Sign Extension Unit converts these immediate fields into 32-bit values before they are used by the processor.

The unit supports:

- I-Type immediates
- S-Type immediates
- B-Type immediates
- U-Type immediates
- J-Type immediates

5.6 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) performs arithmetic, logical, comparison, and address calculation operations required during instruction execution.

The ALU supports operations such as:

- Addition (ADD)
- Subtraction (SUB)
- Bitwise AND
- Bitwise OR
- Bitwise XOR
- Set Less Than (SLT)
- Shift Left Logical (SLL)
- Shift Right Logical (SRL)
- Shift Right Arithmetic (SRA)

The ALU also generates comparison results for branch instructions.

5.7 Data Memory

The Data Memory is used for load and store operations.

During execution:

- Load instructions read data from memory
- Store instructions write data into memory

The memory is accessed using the address generated by the ALU.

Functions:

- Read data from memory
- Write data to memory
- Support load and store instructions

5.8 Control Unit

The Control Unit generates the control signals required for proper instruction execution. It decodes the opcode and function fields of an instruction and determines the operation to be performed by different modules.

The generated control signals include:

- RegWrite
- MemWrite
- MemRead
- ALUSrc
- ResultSrc
- ALUControl
- ImmSrc
- Branch Control Signals
- PCSrc

The Control Unit acts as the decision-making unit of the processor and ensures correct operation of the datapath.

5.9 ALUSrc Multiplexer

The ALUSrc Multiplexer selects the second operand for the ALU. Depending on the instruction type, the operand can be either register data from the Register File or an immediate value generated by the Sign Extension Unit.

Functions:

- Select ALU input source
- Support register and immediate instructions
- Enable address calculation for load and store operations

5.10 Result Multiplexer

The Result Multiplexer selects the data that will be written back to the Register File. Depending on the instruction, the selected result may come from the ALU, Data Memory, immediate value, or PC+4 value.

Functions:

- Select write-back data
- Support arithmetic, memory, and jump instructions
- Control register updates

5.11 WD3 Multiplexer

The WD3 Multiplexer selects the final data input to the write port of the Register File. It enables the processor to write different types of results into destination registers based on the instruction being executed.

Functions:

- Select register write-back source
- Support ALU, memory, immediate, and PC+4 results
- Enable flexible register updates

5.12 PC Multiplexer

The PC Multiplexer determines the next value of the Program Counter. It selects between the sequential address (PC+4) and target addresses generated for branch and jump instructions.

Functions:

- Select next PC value
- Support branching and jumping
- Control program flow

5.13 PC+4 Adder

The PC+4 Adder generates the address of the next sequential instruction by adding 4 to the current Program Counter value. Since each RISC-V instruction occupies 4 bytes, incrementing the PC by 4 allows continuous instruction execution.

5.14 PC Target Adder

The PC Target Adder calculates branch and jump target addresses by adding the sign-extended immediate value to the current Program Counter. The generated address is used whenever a branch or jump instruction is taken.

6. CONTROL SIGNALS

The Control Unit generates various control signals based on the instruction opcode and function fields. These signals control the operation of different datapath components and ensure correct instruction execution.

CONTROL SIGNAL	DESCRIPTION
RegWrite	Enables writing data into the Register File.
MemWrite	Enables write operations to Data Memory during store instructions.
MemRead	Enables read operations from Data Memory during load instructions.
ALUSrc	Selects the second ALU operand between register data and immediate value.
ResultSrc	Selects the data source for register write-back (ALU result, memory data, immediate value, or PC+4).
ALUControl	Determines the operation performed by the ALU.
ImmSrc	Specifies the immediate format generated by the Sign Extension Unit.
Branch	Indicates a branch instruction and enables branch decision logic.
Jump	Indicates a jump instruction and updates the Program Counter accordingly.
PCSrc	Selects the source of the next Program Counter value.
WD3Src	Selects the data source supplied to the Register File write port.

These control signals coordinate the operation of the datapath modules and ensure proper execution of arithmetic, logical, memory access, branch, and jump instructions in the Single-Cycle RISC-V processor.

6.1 Control Signals

The following table shows the control signal values generated by the Control Unit for each instruction supported by the processor.

Instruction	RegWrite	MemWrite	ALUSrc	ResultSrc	PCSrc	ImmSrc	WD3Src	ALUControl
ADD	1	0	0	00	00	XXX	0	0000
SUB	1	0	0	00	00	XXX	0	0001
AND	1	0	0	00	00	XXX	0	0010
OR	1	0	0	00	00	XXX	0	0011
SLT	1	0	0	00	00	XXX	0	0100
XOR	1	0	0	00	00	XXX	0	0101
SLL	1	0	0	00	00	XXX	0	0110
SRL	1	0	0	00	00	XXX	0	0111
SRA	1	0	0	00	00	XXX	0	1000
ADDI	1	0	1	00	00	000	0	0000
ANDI	1	0	1	00	00	000	0	0010
ORI	1	0	1	00	00	000	0	0011
XORI	1	0	1	00	00	000	0	0101
SLTI	1	0	1	00	00	000	0	0100
SLLI	1	0	1	00	00	000	0	0110
SRLI	1	0	1	00	00	000	0	0111
SRAI	1	0	1	00	00	000	0	1000
LW	1	0	1	01	00	000	0	0000
SW	0	1	1	XX	00	001	0	0000
BEQ	0	0	0	XX	Branch Dependent	010	0	0001
BNE	0	0	0	XX	Branch Dependent	010	0	0001
BLT	0	0	0	XX	Branch Dependent	010	0	0100
BGE	0	0	0	XX	Branch Dependent	010	0	0100
LUI	1	0	X	11	00	011	0	XXXX
JAL	1	0	X	XX	01	100	1	XXXX
JALR	1	0	1	00	10	000	1	0000

7. MUX ENCODING

7.1 ResultSrc Encoding

ResultSrc selects the data source that will be written back to the destination register.

VALUE	SOURCE
00	ALU Result
01	Data Memory
10	PC+4
11	Immediate Value

7.2 PCSrc Encoding

PCSrc determines the source of the next Program Counter value for sequential, branch, and jump operations.

VALUE	SOURCE
00	PC+4
01	PC Target (Branch/JAL)
10	ALU Result (JALR)

7.3 ImmSrc Encoding

ImmSrc specifies the instruction format used by the Sign Extension Unit to generate the required immediate value.

VALUE	FORMAT
000	I-Type
001	S-Type
010	B-Type
011	U-Type
100	J-Type

8. INSTRUCTION FORMATS OF RISC-V

RISC-V instructions are categorized into different formats based on their functionality and operand requirements. Each instruction is 32 bits wide and contains fields such as opcode, destination register, source registers, function codes, and immediate values. The RV32I base integer instruction set consists of approximately 40 instructions organized into various instruction formats. Due to project scope and implementation requirements, the proposed Single-Cycle RISC-V processor supports a subset of 26 commonly used instructions, including arithmetic, logical, immediate, load, store, branch, jump, and upper-immediate instructions. These supported instructions are distributed across R-Type, I-Type, S-Type, B-Type, U-Type, and J-Type formats, enabling the processor to efficiently perform arithmetic, logical, memory access, branch, and jump operations.

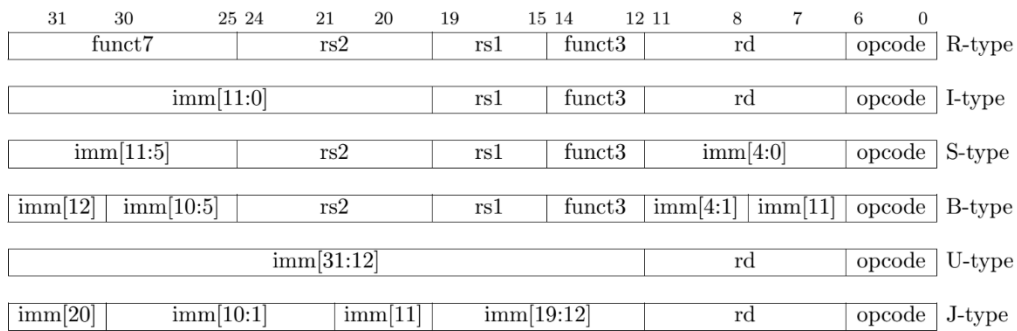


Fig.2: Instruction Format

8.1 R-Type Instructions

R-Type instructions are used for register-to-register arithmetic and logical operations. These instructions use two source registers and one destination register.

Supported R-Type Instructions:

- ADD
- SUB
- AND
- OR
- XOR
- SLT
- SLL
- SRL
- SRA

8.2 I-Type Instructions

I-Type instructions use an immediate value along with a source register. They are commonly used for arithmetic operations, loads, and indirect jumps.

Supported I-Type Instructions:

- ADDI
- ANDI
- ORI
- XORI

- SLTI
- SLLI
- SRLI
- SRAI
- LW
- JALR

8.3 S-Type Instructions

S-Type instructions are used for store operations. The immediate value is split across two fields within the instruction.

Supported S-Type Instructions:

- SW

8.4 B-Type Instructions

B-Type instructions are used for conditional branch operations. The immediate field is arranged differently to support branch target calculation.

Supported B-Type Instructions:

- BEQ
- BNE
- BLT
- BGE

8.5 U-Type Instructions

U-Type instructions use a 20-bit upper immediate value.

Supported U-Type Instructions:

- LUI

8.6 J-Type Instructions

J-Type instructions are used for unconditional jump operations.

Supported J-Type Instructions:

- JAL

9. DATAPATH OPERATION FOR INSTRUCTIONS

The Single-Cycle RISC-V processor executes all instructions using a common datapath. Depending on the instruction type, different datapath components such as the Register File, ALU, Data Memory, multiplexers, and Program Counter are activated through control signals generated by the Control Unit. The following sections describe the flow of data through the processor for various instruction categories.

9.1 R-Type Instruction Execution

For R-Type instructions, the source operands are read from the Register File and supplied to the ALU. The ALU performs the required arithmetic or logical operation and generates the result. The Result Multiplexer selects the ALU output, which is then written back to the destination register.

Datapath Flow:

PC → Instruction Memory → Register File → ALU → Result MUX → Register File

9.2 I-Type Instruction Execution

For I-Type instructions, the immediate value extracted from the instruction is sign-extended to 32 bits. The ALUSrc Multiplexer selects the immediate value as the second ALU operand. The ALU performs the required operation and the result is written back to the destination register.

Datapath Flow:

PC → Instruction Memory → Register File + Sign Extension Unit → ALU → Result MUX → Register File

9.3 Load and Store Instruction Execution

For memory access instructions, the ALU calculates the effective memory address using the base register value and sign-extended immediate value.

During a load operation, data is read from Data Memory and written back to the Register File. During a store operation, data from the Register File is written into Data Memory.

Datapath Flow:

PC → Instruction Memory → Register File → ALU → Data Memory → Result MUX → Register File

9.4 Branch Instruction Execution

For branch instructions, the Register File supplies two operands for comparison. The ALU evaluates the branch condition and generates the corresponding comparison result. The Control Unit uses this result to determine whether the branch target address or the next sequential address should be loaded into the Program Counter.

Datapath Flow:

PC → Instruction Memory → Register File → ALU Comparison → Branch Decision Logic → PC Target Adder → PC

9.5 Jump Instruction Execution

For jump instructions, the target address is calculated either using the PC Target Adder or the ALU. The Program Counter is updated with the target address through the PC Multiplexer. Simultaneously, the return address (PC + 4) is written back to the destination register.

Datapath Flow:

PC → Instruction Memory → Address Calculation Unit → PC MUX → PC

Return Address Flow:

PC → PC+4 Adder → WD3 MUX → Register File

10. TESTBENCH DESIGN

A Verilog testbench was developed to verify the functionality and correctness of the Single-Cycle RISC-V processor. The testbench generates the clock signal required for synchronous processor operation and applies an initial reset signal to initialize the Program Counter and other processor components before instruction execution begins.

The Instruction Memory is initialized using an external program.mem file containing machine code instructions. During simulation, the processor fetches instructions sequentially from this memory and executes them according to the implemented control logic and datapath design.

Various test programs were executed to validate the operation of arithmetic, logical, immediate, load, store, branch, and jump instructions. The simulation monitored the contents of registers, data memory locations, ALU outputs, control signals, and Program Counter updates to ensure correct instruction execution. Different scenarios, including memory accesses, conditional branching, jump operations, and register write-back operations, were verified to confirm the proper functioning of the processor. The successful execution of these test cases demonstrated the correctness of both the datapath and control unit implementations.

This code along but different cases was tested:

ASSEMBLY CODE	MACHINE CODE
ADDI x1, x0, 5	00500093
ADDI x2, x0, 10	00A00113
ADD x3, x1, x2	002081B3
SUB x4, x2, x1	40110233
AND x5, x1, x2	0020F2B3
OR x6, x1, x2	0020E333
XOR x7, x1, x2	0020C3B3
SLT x8, x1, x2	0020A433
SLL x9, x1, x2	002094B3
SRL x10, x2, x1	00115533

11. SIMULATION RESULTS

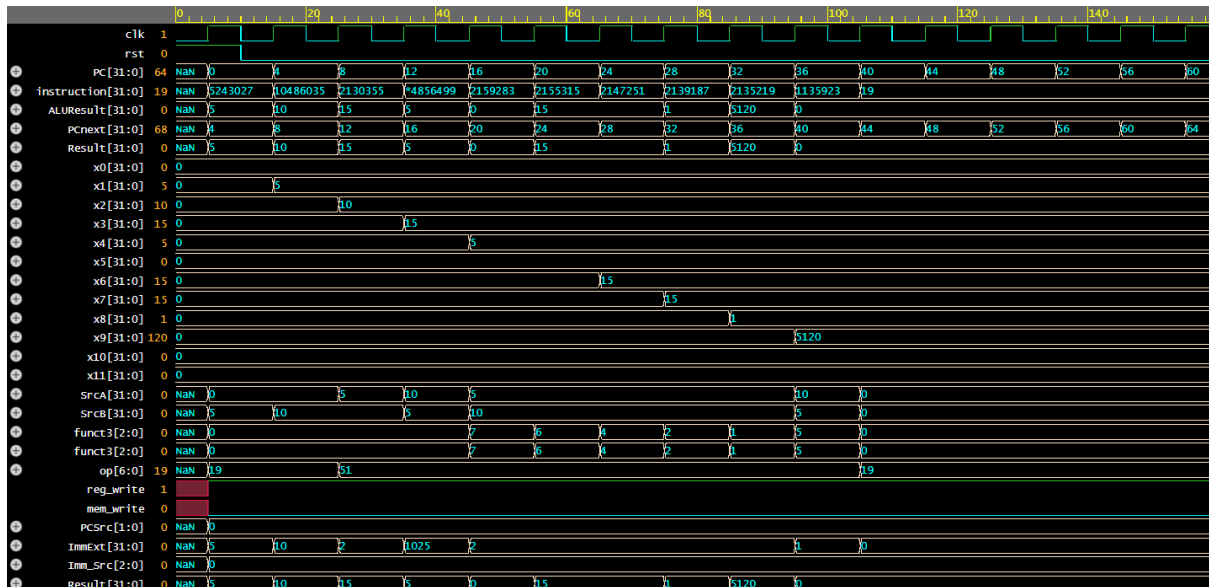


Fig.2: Simulation Result

REGISTER	VALUE
x1	5
x2	10
x3	15
x4	5
x5	0
x6	15
x7	15
x8	1
x9	5120
x10	0

The observed register values match the expected results, confirming the correct operation of the ALU, Register File, Control Unit, and datapath components.

12. COMPARISON WITH MULTICYCLE AND PIPELINE

Processor architectures can be implemented using Single-Cycle, Multi-Cycle, or Pipelined designs. Each approach differs in terms of instruction execution, hardware complexity, and overall performance. The Single-Cycle RISC-V processor implemented in this project executes every instruction in a single clock cycle, making the design simple and easy to understand. However, this simplicity comes at the cost of lower performance compared to more advanced processor architectures.

FEATURE	SINGLE-CYCLE PROCESSOR	MULTI-CYCLE PROCESSOR	PIPELINED PROCESSOR
Instruction Execution	One cycle per instruction	Multiple cycles per instruction	Overlapped execution of instructions
Clock Period	Long	Shorter	Short
Hardware Complexity	Low	Medium	High
Control Unit Design	Simple	Moderate	Complex
Throughput	Low	Moderate	High
Hardware Utilization	Lower	Better	Highest
Hazard Handling	Not Required	Not Required	Required
Performance	Lower	Moderate	High

The Single-Cycle processor offers a simple datapath and control structure, making it suitable for learning and understanding processor fundamentals. Multi-Cycle processors improve hardware utilization by dividing instruction execution into multiple stages. Pipelined processors further enhance performance by executing multiple instructions simultaneously at different stages, resulting in higher throughput and better overall efficiency. However, pipelined architectures require additional hardware and hazard management mechanisms, increasing design complexity.

13. ADVANTAGES

- Simple and easy-to-understand processor architecture.
- All instructions are completed in a single clock cycle.
- Control unit design is straightforward and less complex.
- Easy to verify and debug during simulation.
- Provides a strong foundation for learning advanced processor architectures.

14. LIMITATIONS

- Clock period is determined by the slowest instruction.
- Lower performance compared to pipelined processors.
- Inefficient hardware utilization for simple instructions.
- Increased clock cycle time due to complete instruction execution in one cycle.
- Not suitable for high-performance computing applications.

15. APPLICATIONS

- Educational and academic processor design projects.
- Learning and research in computer architecture.
- Foundation for developing multi-cycle and pipelined processors.
- Embedded system prototyping and experimentation.

16. CHALLENGE FACED

- Designing the datapath to support multiple instruction formats.
- Generating correct control signals for different instruction types.
- Implementing branch and jump operations correctly.
- Verifying memory read and write operations during simulation.
- Debugging instruction execution and register updates.

17. LEARNING OUTCOMES

- Gained understanding of the RISC-V RV32I instruction set architecture.
- Learned the implementation of a Single-Cycle processor using Verilog HDL.
- Understood datapath design and control unit operation.
- Acquired knowledge of instruction execution and memory interfacing.

18. FUTURE ENHANCEMENTS

- Extend support to the complete RV32I instruction set.
- Implement a Multi-Cycle RISC-V processor architecture.
- Upgrade the design to a Pipelined RISC-V processor.
- Add support for interrupts and exception handling.
- Integrate cache memory and advanced RISC-V extensions.

19. CONCLUSION

This project successfully implemented a 32-bit Single-Cycle RISC-V processor based on the RV32I instruction set using Verilog HDL. The processor was designed using essential modules such as the Program Counter, Instruction Memory, Register File, Control Unit, ALU, Data Memory, and supporting datapath components. The implemented design supports arithmetic, logical, immediate, load, store, branch, and jump instructions within a single clock cycle. Simulation results verified the correct operation of all supported instructions, control signals, memory accesses, and program flow operations. The project provided valuable insight into processor architecture, datapath design, and instruction execution, while also establishing a strong foundation for the development of more advanced multi-cycle and pipelined RISC-V processors.

20. REFERENCES

- [1] Computer Organization and Design RISC-V Edition, D. A. Patterson and J. L. Hennessy.
- [2] RISC-V International, "RISC-V Specifications".
- [3] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Pearson Education.
- [4] M. Morris Mano and Michael D. Ciletti, *Digital Design*, Pearson Education.